



무인편의점 관리자

C언어 선택형 시뮬레이션 게임

C언어 알고리즘 실습 · 기말 프로젝트

박수용 2601110288

1 주제 선정 배경
왜 이 게임을 만들었는지

2 프로젝트 개요
어떤 게임인지

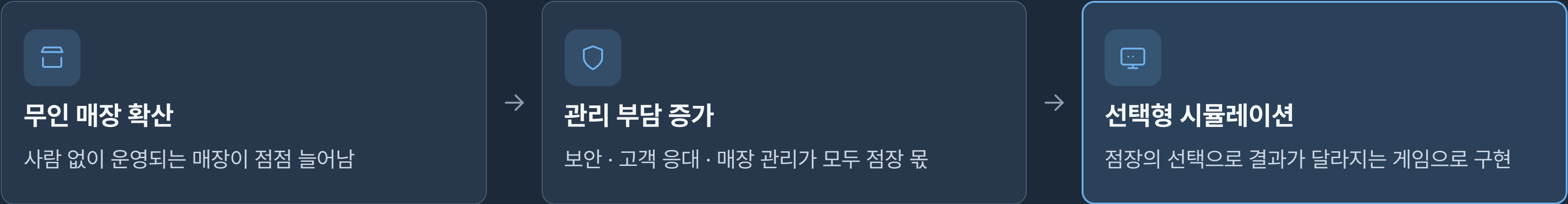
3 게임 진행 방식
진행 흐름과 상태 지표

4 핵심 구현 내용
구조체 · 셔플 · 입력 검증

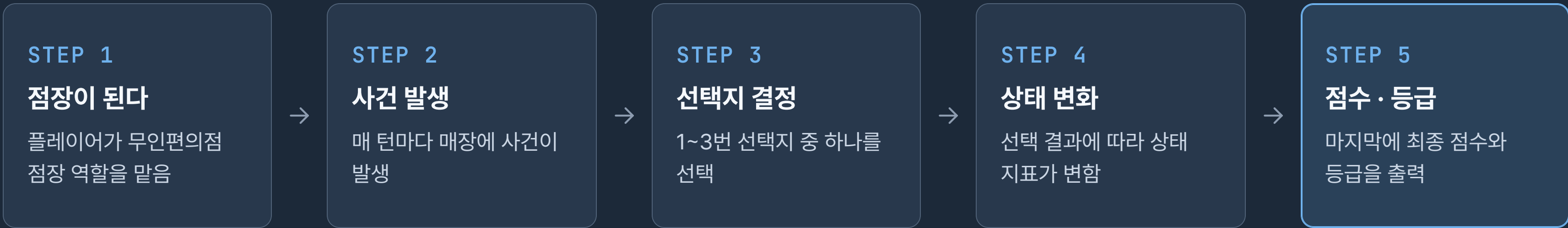
5 실행 결과
실제 콘솔 실행 화면

6 느낀 점
어려웠던 점과 배운 점

무인 매장이 늘어나면서 **보안, 고객 응대, 매장 관리**가 점점 중요해졌다고 생각했습니다.
그래서 점장의 선택에 따라 결과가 달라지는 **시뮬레이션 게임**으로 만들어 보았습니다.



플레이어는 무인편의점 점장이 되어, 매 턴 발생하는 사건에 선택으로 대응합니다.



프로그램은 다음 순서로 진행됩니다. 핵심은 가운데의 **5턴 반복 루프**입니다.



선택 결과는 네 가지 지표에 반영되고, 게임이 끝나면 이 값들로 최종 점수를 계산합니다.



보안 점수

매장 감시와 대응을 잘할수록 올라감

높을수록 좋음



매출

고객 응대를 잘할수록 늘어남

높을수록 좋음



평판

친절하고 빠른 대응으로 쌓임

높을수록 좋음



절도 위험

방심하면 빠르게 올라감

낮을수록 좋음

최종 점수 = **보안** + **매출** + **평판** - **절도 위험**

매 턴 발생하는 이벤트는 세 가지 범주로 나누어 만들었습니다.



보안 이벤트

- 절도 의심 행동
- 수상한 방문자



고객 이벤트

- 결제 오류
- 고객 불만



매장 관리 이벤트

- 재고 부족
- 냉장고 고장

Player 구조체를 사용해 플레이어의 상태를 하나로 묶어 관리했습니다.

```
player.h

// 플레이어 상태를 담는 구조체
typedef struct {
    char name[20];    // 이름
    int  security;    // 보안 점수
    int  sales;        // 매출
    int  reputation;   // 평판
    int  risk;         // 절도 위험
    int  score;        // 종합 점수
} Player;
```

- 여러 상태값을 구조체 하나로 묶어 관리했습니다.
- 함수에는 포인터(**Player *p**)로 전달했습니다.
- 덕분에 함수 안에서 값을 직접 수정할 수 있습니다.

이벤트가 매번 같은 순서로 나오면 단조롭기 때문에, **Fisher-Yates 셔플**로 순서를 섞었습니다.

원래 순서

E1

E2

E3

E4

E5



섞인 순서

E3

E1

E5

E2

E4

```
for (int i = n-1; i > 0; i--) {  
    int j = rand() % (i+1);  
    // a[i] 와 a[j] 를 교환  
    int t = a[i];  
    a[i] = a[j];  
    a[j] = t;  
}
```

- 배열을 한 번 순회하며 무작위 위치와 교환
- 시간복잡도 $O(n)$ · 중복 없이 순서만 변경

잘못된 입력으로 프로그램이 멈추지 않게 막고, 최고 점수는 파일에 저장되도록 했습니다.

잘못된 입력 처리

● ● ● 입력 검증

선택 (1/2/3): a
[오류] 숫자만 입력하세요.

선택 (1/2/3): 5
[오류] 1~3만 선택 가능합니다.

선택 (1/2/3): 2
→ 정상 처리

최고 점수 저장

● ● ● highscore.txt

[최고 점수]
플레이어 : 박수용
점수 : 260

→ 새로운 최고 점수 저장 완료!
(highscore.txt 에 기록됨)

- scanf 반환값 확인 · 문자 입력 오류 처리
- 1~3 범위만 허용
- 최고 점수 자동 저장

메인 메뉴

```
=====
[ 무인편의점 관리자 ]
=====
1. 게임 시작
2. 게임 설명
3. 최고 점수 확인
4. 종료
=====
메뉴 선택: 1
이름은 입력하세요. <반송>
```

이벤트 선택

```
[ 1번째 상황 / 5턴 ]
[상황] 손님이 상품을 들고 결제하지
않은 채 출구로 향합니다.

1. 바로 경찰에 신고한다.
2. 경고 방송을 한다. ← 선택
3. 조금 더 지켜본다.

→ 경고로 상황을 해결했습니다.
```

최종 결과

```
=====
[ 최종 결과 ]
=====
보안 점수 : 85      매출 : 120
평판      : 65      절도 위험 : 10

최종 점수 : 260      등급 : S
=====
```

어려웠던 점

01 랜덤 이벤트 중복 방지

02 구조체 포인터 사용

03 잘못된 입력 처리

느낀 점

이번 프로젝트를 통해 구조체, 포인터, 배열, 조건문, 반복문, 파일 입출력을 실제 게임 로직에 적용해 볼 수 있었습니다.

단순히 코드를 작성하는 것보다, 전체 흐름을 설계하고 오류 상황을 처리하는 과정이 중요하다는 것을 느꼈습니다.

감사합니다.